Aakash Dandekar

CS Undergrad · Seeking Data Science | AI/ML | SWE Internship

aakashdandekar2006@gmail.com | github.com/aakashdandekar | linkedin.com/in/aakash-dandekar-6055a5317 | leetcode.com/aakashdandekar

EDUCATION

B.Tech / B.E. in Computer Science & Engineering **Parul University, Gujarat, India · 2024 – Present**

Relevant Coursework: Data Structures & Algorithms, Python, Databases, OOP, Maths for CS, Computer Networks

Building full-stack, backend, and data-oriented projects independently alongside academics.

CERTIFICATIONS

IBM RAG and Agentic AI Professional Certificate · **Feb 2026**

IBM

Machine Learning Specialization · **Feb 2026**

Stanford Online / DeepLearning.AI (Coursera) · 3 Courses

Python · **Sep 2025**

IBM Developer Skills Network / Etrain Education (IBMCE)

TECHNICAL SKILLS

Languages Python · JavaScript · Go · SQL · Shell

AI / ML LangChain · HuggingFace · ChromaDB / FAISS · Groq Whisper · NumPy · Pandas · Scikit-learn · Matplotlib · Jupyter · PyTorch

Development FastAPI · Flask · Django · Next.js · Vue.js · MongoDB · SQLite · Motor (async) · Pydantic · Uvicorn · Cryptography · JWT · bcrypt · ImageKit

Tools & Infra Git · GitHub Actions · OpenEnv · Ollama · Docker · Supabase · Firebase

PROJECTS

DebateX — **Python · Vue.js · FastAPI · Groq API · LangChain · ChromaDB · HuggingFace MiniLM · MongoDB · JWT**

- Built a production-grade async debate platform where users argue against LLMs with an impartial AI judge delivering quantitative scoring, structured feedback, and a final verdict via Groq-hosted models.
- Implemented a three-model Groq pipeline: llama-3.1-8b-instant for topic generation, llama-3.3-70b-versatile for counter-argument synthesis, and gpt-oss-120b for contextual adjudication — demonstrating end-to-end multi-model orchestration.
- Integrated ChromaDB with HuggingFace MiniLM embeddings for semantic context retrieval; secured 10+ routes with JWT + Bcrypt authentication and maintained full per-session conversation history.

Lexify — **Python · FastAPI · MongoDB · LangChain · Sentence-Transformers · ChromaDB/FAISS · Fernet · JWT**

- Built an AI-powered legal document analysis platform; users upload PDF/DOCX contracts to receive automated context summaries, clause-by-clause plain-language explanations, and RAG-powered Q&A.;
- Implemented vector search using Sentence-Transformers + ChromaDB/FAISS for semantic retrieval; secured sensitive legal data end-to-end with Fernet symmetric encryption, JWT authentication, and bcrypt password hashing.
- Delivered persistent chat sessions with auto-generated titles and retrievable history; full-stack interface auto-documented via Swagger UI / ReDoc.

Social Media WebApp — **Python · FastAPI · MongoDB (Motor async) · ImageKit CDN · JWT · Pydantic · Uvicorn**

- Built a RESTful backend API for a social media platform supporting user authentication, image/video post uploads via ImageKit CDN, likes, comments, follow/unfollow, and user search across 14 endpoints.
- Implemented JWT-based access tokens (2-hour expiry), bcrypt password hashing, and a randomized feed algorithm that excludes the authenticated user's own posts.
- Structured with clean module separation (auth, db, schemas, routes); CORS-ready for frontend integration at localhost:5173; full async MongoDB queries via Motor driver.

Agentix — **Python · Ollama · Shell · Linux**

- Built a lightweight agentic CLI AI assistant supporting any tool-capable LLM via Ollama, with real tool-use: run shell commands, read/write files, browse the web, and list directories.
- Packaged as an installable CLI tool via pyproject.toml with timestamped JSONL audit log, persistent config at ~/.aiagent/config.yaml, and safe-mode sandboxing for all file/shell operations.

Goal Tracker — **Python · Flask · SQLite · JavaScript · CSS · HTML**

- Built a full-stack goal-tracking web app with a Flask backend, SQLite persistence, and a dynamic JavaScript frontend featuring real-time goal creation, status toggling, and progress tracking.
- Implemented modular architecture with separate database (database.py) and business logic (operations.py) layers; clean responsive UI using vanilla CSS and JS without any frontend framework.

SQL Debug Environment — **Python · FastAPI · SQLite · Docker · OpenEnv · HuggingFace Spaces**

- Designed an OpenEnv-compliant RL benchmark where AI agents iteratively debug broken SQL queries across 9 hand-crafted tasks spanning easy (syntax), medium (logic), and hard (CTE/window functions) difficulty tiers.
- Built a deterministic grader using in-memory SQLite with shaped rewards (0.0–1.0); baseline GPT-4o-mini achieved 78% / 51% / 24% success on easy/medium/hard — establishing a reproducible public benchmark.
- Containerised with Docker and deployed to HuggingFace Spaces; exposes step / reset / state / grade REST endpoints with reference inference.py demonstrating full LLM agent integration.